

TITLE : LEXICAL ANALYZER Using C

This project implements the first two phases of a compiler: lexical analysis and syntax analysis. The lexical analyzer scans the source program and converts it into a sequence of tokens such as keywords, identifiers, operators, numbers, special symbols, and preprocessor directives while ignoring comments and whitespace. The syntax analyzer then validates the source program by checking for common syntax errors, including missing semicolons, missing identifiers, unmatched parentheses, and unmatched braces. The project demonstrates the basic working principles of compiler front-end design using C programming and modular programming concepts.

TOPICS USED IN THIS PROJECT:

- Compiler Design (Lexical & Syntax Analysis)
- C Programming
- File Handling
- String Handling (string.h)
- Character Handling (ctype.h)
- Functions
- Arrays
- Conditional Statements
- Loops
- Modular Programming
- Command-Line Arguments (argc, argv)

1. LEXER.H

Description:

It is the header file of the lexical analyzer. It contains the function prototypes required for token classification, such as checking whether a token is a keyword, operator, special symbol, number, or identifier. It also declares the lexicalAnalyzer() function, which performs tokenization of the source file. Including this header allows these functions to be accessed from other source files without rewriting their declarations.

2. LEXER.C

Description:

It implements the lexical analyzer, which is the first phase of the compiler. It reads the source program character by character and groups them into meaningful tokens. The file contains functions to identify keywords, identifiers, numbers, operators, and special symbols using character classification and string comparison techniques. It ignores whitespace and single-line comments, recognizes preprocessor directives, and prints the type of each token found in the source file. This module converts the input program into a sequence of tokens that can later be processed by the syntax analyzer.

3. SYNTAX_ANALYZER

Description:

It is the header file for the syntax analyzer. It declares the `syntaxanalyzer()` function, which is responsible for checking basic syntax errors in the source program. This header allows the syntax analyzer module to be called from the main program.

4. SYNTAX_ANALYZER.C

Description:

It implements the syntax analyzer, which is the second phase of the compiler. It reads the source code line by line and performs simple syntax validation. The module detects missing semicolons, missing identifiers in variable declarations, unmatched parentheses in control statements (`if`, `else`, `for`, `while`), and unmatched curly braces. It also removes single-line comments before checking syntax to avoid false errors. Whenever an error is detected, the corresponding line number and error message are displayed. If no syntax errors are found, it reports that the source code is syntactically correct according to the implemented checks.

5. MAIN.C

Description:

It is the driver program of the project. It accepts the source file name as a command-line argument and opens the file in read mode. The lexical analyzer is executed first to generate tokens from the source program. After lexical analysis, the file pointer is reset using `rewind()` so that the syntax analyzer can process the same file from the beginning. Finally, the file is closed after both analysis phases are completed. This file integrates all the modules and controls the execution flow of the project.